Lab day: SO-101 arm + SmolVLA on the Jetson

Thursday 2026-10-22, ENG276. Lam Nguyen, Vi Thi Tuong Nguyen, James Pham

Goals for the day

1. Check the lab arm hardware and write down what we still need to order.
2. Run a baseline VLA, SmolVLA, on the Jetson and let it drive the real arm.
3. If the lab has a leader arm, record demos so we can fine-tune SmolVLA for a real task.

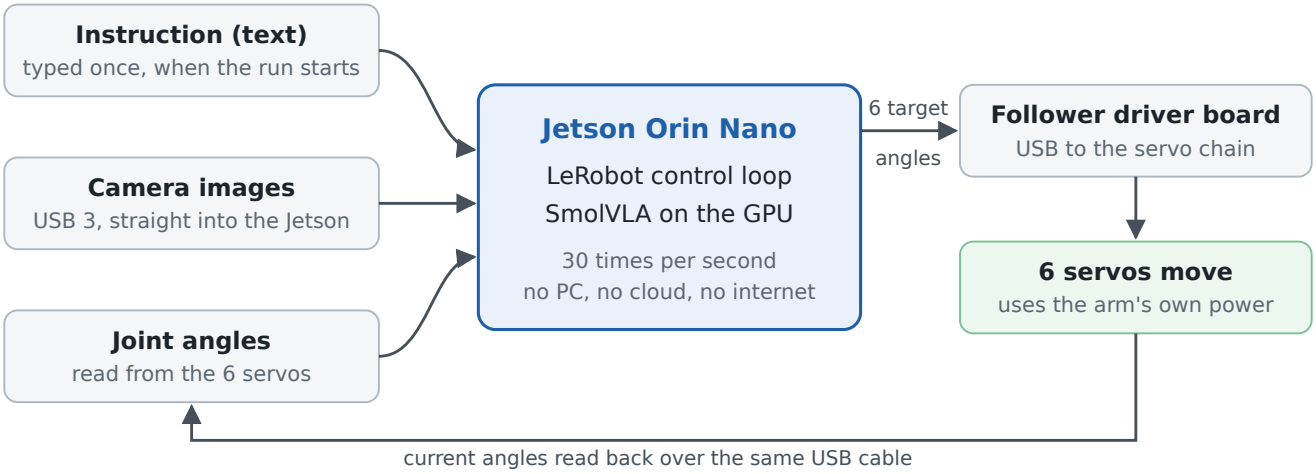
Software on the Jetson is ready. SmolVLA and its backbone are installed and cached, and they work with no internet. Every command below runs from `~/edge-vla` in a normal terminal. You don't need to activate anything first.

Contents

- Part 1. How the Jetson, SmolVLA and the arm work together
- Part 2. Before we leave home
- Part 3. At the lab, step by step
  - 1. Set up the Jetson
  - 2. Look at the arms first
  - 3. Connect the follower
  - 4. Find the follower's port
  - 5. Check the follower's servos
  - 6. Connect and check the leader
  - 7. Calibrate both arms
  - 8. Teleop test
  - 9. Cameras
  - 10. Baseline: SmolVLA drives the arm
  - 11. Record demos
  - 12. Pack up
- Part 4. Inventory and order list
- Part 5. Troubleshooting
- Part 6. After Thursday
- Numbers measured on our board

Part 1. How the Jetson, SmolVLA and the arm work together

The pieces



- **Follower arm:** the robot that does the work. It has a gripper with two jaws and 6 servo motors, IDs 1 to 6: shoulder pan, shoulder lift, elbow, wrist flex, wrist roll, gripper.
  - Each servo knows its own angle and has a small built-in controller that drives the motor to a target angle.
- **Driver board:** a small board on the arm. It connects the servo chain to the Jetson through one USB cable, which shows up on the Jetson as `/dev/ttyACM0`.
  - The servos get their power from the arm's own power supply, not from USB.
- **Leader arm:** the same shape, but with a handle and trigger instead of a gripper, and its motors stay off. You move it by hand, and the follower copies it.
  - We use it only for teleop and for recording demos. The policy doesn't need it.
- **Cameras:** the policy's eyes. They connect over USB 3, directly into the Jetson.
- **Jetson:** runs everything. LeRobot reads the arm and cameras, SmolVLA decides the next moves on the GPU, and LeRobot sends the moves to the servos.
  - There is no PC, no cloud, and no internet in the loop.

What happens when SmolVLA drives the arm, step by step

When you start `bash hardware/so101_lab.sh policy test1`:

- It takes about 15 s to load SmolVLA from the Jetson's disk onto the GPU.
- It connects to the follower and turns its motors on. The arm stiffens and holds its pose.
- It opens the cameras.
- Then it repeats this loop **30 times per second**, about every 33 ms:

1. **Read the arm.** LeRobot asks all 6 servos for their current angle over the USB cable, which takes about 2 ms. The 5 joints come back in degrees, and the gripper from 0 (closed) to 100 (open).
2. **Read the cameras.** It takes the newest picture from each camera, 640x480 in color.

3. **Plan, if needed.** SmolVLA keeps a queue of 50 planned moves. When the queue is empty, it plans 50 new ones:
   - Each picture is resized to 512x512 and turned into 64 "image tokens" by a vision encoder.
   - The instruction text is cut into up to 48 word-piece tokens.
   - The 6 joint angles become one "state token".
   - A small language model, the first 16 layers of SmolVLM2, reads all these tokens together.
   - An "action expert" starts from random noise and cleans it up in 10 steps into 50 future moves of 6 numbers each. This method is called flow matching.
   - On our Jetson, planning takes about **1.1 s**. No new commands go out during that time, so **the arm holds still**.
4. **Send one move.** LeRobot takes the next of the 50 moves and limits each joint to at most **3 degrees** away from where it is now. This is our safety cap, `MAX_STEP`. Then it writes the 6 target angles to the servos.
5. **Save the step** to an "eval" dataset, so we can replay the run later.
6. **Wait** until the 33 ms step is over, then go back to step 1.

So the arm moves in bursts:

plan about 1.1 s, still	50 moves 1.7 s, moving	plan about 1.1 s, still	50 moves 1.7 s, moving	...

`STEPS=5` cuts the planning time to about 0.7 s. Making it smooth is exactly what our project is about.

Do you type English and the arm does it?

Partly. Here is what really happens:

- **You type the instruction once, before the run:** `TASK="Put the red cube in the box."`
  - It stays the same for the whole run.
  - You can't talk to the arm while it moves. A new instruction means a new run.
- **The model reads the instruction every time it plans**, together with the camera pictures and the joint angles.
  - It does not split the sentence into steps like "find red", "grab" and "move".
  - The words are one input that steers which motion it produces, based on examples it has seen.
- **It can only do what it learned from examples.**
  - After we fine-tune it on about 50 demos of "Put the red cube in the box.", it will do that motion when the scene looks like the demos.
  - Use the same sentence as in the demos.
  - A new sentence, like "Stack the blocks", will not make it stack blocks if it never saw stacking.
- **The words pick between tasks it knows.** If we record demos for 2 or 3 tasks, each with its own sentence, the sentence chooses the task. That is the "language" part of a vision-language-action model.

- **It keeps watching.** It plans again every 1.7 s from fresh pictures, so if the cube moves, the next plan goes to the new spot.
- **On Thursday the base model has never seen this arm, this camera angle or this table.** It will move the arm, but it won't do the task. That's expected. Thursday proves that the whole loop works.

How demos teach it

- During recording, you move the **leader** by hand, and the follower copies it 30 times per second.
- Every saved step holds:
  - what the robot saw and felt: the camera pictures and the follower's joint angles
  - what the human did next: the leader's angles
  - the instruction
- Fine-tuning shows SmolVLA thousands of these steps until it can predict the human's next 50 moves from the pictures, the angles and the sentence.
- We'll do the fine-tuning on the RTX 4090 after Thursday.

Part 2. **Before we leave home**

- [ ] **Serial port access (one sudo step, James).** Without it, nothing can talk to the arm. Run this, then log out and back in. Ask Dr. Liu first if we keep our "admin changes need his OK" rule.

  ```
  sudo usermod -aG dialout $USER
  ```

- [ ] **A camera.** We have none. Ask whether the lab arm has cameras.
  - If it doesn't, buy at least one **USB 3.0** UVC camera before Thursday.
  - Without a camera the policy can't run, and demos are useless.
- [ ] **If we have the camera at home,** plug it into the Jetson and run `bash hardware/so101_lab.sh check`. It should show the camera at **5000M**. Then try `bash hardware/so101_lab.sh preview <its path>`.
- [ ] Run `bash hardware/so101_lab.sh check`. Only "serial ports" (and "cameras", if we have none) should be missing.
- [ ] Shut the Jetson down from the menu before unplugging it.

Bring

- [ ] The Jetson and its power supply.
- [ ] A DisplayPort cable. The Jetson has **no HDMI port**, so also bring a DisplayPort-to-HDMI cable in case the lab monitor only has HDMI.
- [ ] A keyboard, a mouse, and a **small USB hub**. The Jetson has 4 USB ports, and 2 arms plus 2 cameras fill them all.
- [ ] The camera or cameras, a way to hold each one (a small tripod or clamp), and tape.
- [ ] A USB flash drive, to take the demos and calibration files home.
- [ ] Task objects: 2 or 3 small cubes, about 2 to 3 cm, in different colors, and a box or bowl.
- [ ] A power strip. The Jetson, the monitor and 2 arm power supplies need 4 outlets.

Part 3. **At the lab, step by step**

1 Set up the Jetson

5 min

1. Plug in the monitor (DisplayPort), the hub with the keyboard and mouse, and then the Jetson's power. Log in.
2. Open a terminal and run:

   ```
   cd ~/edge-vla
   bash hardware/so101_lab.sh check
   ```

   The "python env" section should show `cuda True` and two `cached:` lines. Serial ports and cameras will be empty for now.
3. Close Chrome, VS Code and other desktop apps. This frees memory (the board was already using swap) and CPU for the cameras and for saving demos.
   - It won't make SmolVLA plan faster. We tested that (see the numbers at the end).
   - Optional, 1 minute: with everything closed, run `vla`, then `python3 hardware/hello_smolvla.py`. Compare the chunk time with the numbers at the end.

2 Look at the arms before plugging anything in

10 min

Fill in the table in Part 4 as you go.

1. **Which arm is which:**
   - The follower has a **gripper with two jaws**.
   - The leader has a **handle with a trigger**.
2. **Count the servos:** 6 per arm, with 3-wire cables from servo to servo, and a **driver board** near the base (with a USB port, a power jack and servo plugs).
3. **Driver board jumpers:** if it's a Waveshare board, both jumpers must sit on **B** (USB).
4. **Power supplies:** one per arm. Read the voltage on a servo label and on the supply. A 7.4 V servo needs a **5 V** supply, and a 12 V servo needs a **12 V** supply.

   Never connect a 12 V supply to 7.4 V servos.
5. **Clamps:** each arm must be clamped to the table. The base tips over when the arm swings.
6. **Ask the lab:**
   - Are there calibration files from the student who used this arm before?
   - Are there cameras and camera mounts?
   - Is it OK to recalibrate the arm?

3 Connect the follower

2 min

1. Plug the follower's power supply into the wall, then into the follower's driver board. A light on the board turns on.
2. Connect the driver board to a Jetson USB port.
3. Plug in the **follower first**, so it becomes `/dev/ttyACM0`, the default.

4 Find the follower's port

2 min

```
bash hardware/so101_lab.sh ports
```

It lists the ports and asks you to unplug the arm's USB cable and press Enter. Then it prints the arm's port. Plug the cable back in.

- If the port is `/dev/ttyACM0`, you're set.
- If it's something else, run `export FOLLOWER_PORT=/dev/ttyACMx` once in this terminal (use the real number).
- If you see `permission denied`, the dialout step wasn't done. For today, run `sudo chmod 666 /dev/ttyACM0`. You have to repeat it after every replug.

5 Check the follower's servos

1 min, nothing moves

```
bash hardware/so101_lab.sh motors follower
```

This only reads from the servos. It writes nothing, moves nothing, and works before calibration. A good result looks like:

```
  ID 1 shoulder_pan   STS3215   position 2048 of 4095    5.0 V   27 C   torque off
  ...
  6 of 6 servos answer
  supply about 5.0 V: a 5 V class supply. Right for 7.4 V servos, wrong for 12 V servos
```

- **6 of 6, all STS3215, near room temperature:** the arm is healthy.
- **The supply voltage must match the servo type** from Step 2.
- **Anything else:** see Part 5, Troubleshooting.

6 Connect and check the leader

3 min

1. Plug in the leader's own power supply and its USB cable. It should become `/dev/ttyACM1`. Run `ports` again if you're not sure, and if needed `export LEADER_PORT=/dev/ttyACMx`.
2. Run:

   ```
   bash hardware/so101_lab.sh motors leader
   ```

3. **No leader in the lab?** Skip Steps 8 and 11 (no teleop and no demos), and put a leader on the order list.

7 Calibrate both arms

5 min each

Calibration teaches LeRobot where each joint's middle and limits are.

If the lab gives us calibration files, copy them to these paths and skip the rest of this step. When a command later asks, press Enter to use them:

- follower: `~/.cache/huggingface/lerobot/calibration/robots/so_follower/lab_follower.json`
- leader: `~/.cache/huggingface/lerobot/calibration/teleoperators/so_leader/lab_leader.json`

Otherwise, calibrate. Calibrating writes new zero points into the servos, so make sure the lab is OK with it first.

```
bash hardware/so101_lab.sh calibrate follower
```

1. It says **"Move ... to the middle of its range of motion and press ENTER"**.
   - The motors are off, so the arm moves freely by hand.
   - Put every joint roughly in the middle of its travel, then press Enter.
2. It says **"Move all joints except 'wrist_roll' sequentially through their entire ranges of motion"**.
   - Move each joint slowly from one end to the other, one joint at a time. Include the gripper: fully open, then fully closed.
   - The numbers on screen change as you go. Press Enter when you're done.
3. It saves the calibration file.

Then do the same for the leader:

```
bash hardware/so101_lab.sh calibrate leader
```

8 Teleop test

5 min, needs the leader

```
bash hardware/so101_lab.sh teleop
```

1. Move the leader by hand, and the follower copies it. Try each joint slowly, then the gripper.
2. **To stop:** hold the follower with one hand, then press Ctrl-C. Its motors switch off when the program ends, and the arm drops.
3. If a joint moves the wrong way, jumps, or stops short, redo that arm's calibration (Step 7).

9 Cameras

15 min

1. Plug each camera into **one of the Jetson's own USB ports**, not the hub.
2. Run:

   ```
   bash hardware/so101_lab.sh check
   ```

   The cameras section shows each camera's path and its link speed.

- **5000M** means USB 3, which is good.
- **480M** means the wrong port, a USB 2 cable, or a USB 2 camera.

3. **Aim camera 1:** a front or top view that sees the arm, the cube area and the box all at once.

   ```
   bash hardware/so101_lab.sh preview /dev/v4l/by-id/usb-...-video-index0
   ```

   Close the window when it looks right.
4. **Camera 2 (optional):** on the wrist mount, or a side view.
5. **Tape everything down:** the camera mounts, the arm base, the box, and the area where the cube starts. Every demo and every test must look the same to the cameras.
6. **Save the paths** for this terminal, so you don't have to retype them:

   ```
   export CAM1=/dev/v4l/by-id/usb-...-video-index0
   export CAM2=/dev/v4l/by-id/usb-...-video-index0
   ```

7. **Optional frame-rate check:** run `vla`, then `python3 hardware/hello_camera.py --device $CAM1`. It should reach 30 fps.

10 Baseline: SmolVLA drives the follower

5 min

1. Put the arm in a resting pose, and clear the table except for the cube and the box.
2. **One person keeps a hand on the follower's power plug.** Unplugging it stops the arm instantly.
3. Run:

   ```
   TASK="Put the red cube in the box." bash hardware/so101_lab.sh policy test1
   ```

What you will see:

- About 15 s of loading, then the arm stiffens.
- Then it moves in bursts for 20 s: 1.7 s of motion, then about 1.1 s still while the Jetson plans. Moves are slow because of the 3 degree cap.
- The terminal prints "Record loop is running slower ... than the target FPS". **That's normal:** it's the planning pause.

When the motion stops, it saves the run for about 30 s while the arm still holds its pose. Then the motors switch off and **the arm drops, so hold it.**

What counts as success today: the run finishes with no errors and the arm follows the model. **It will not do the task.** The base model has never seen this setup.

Try shorter pauses:

```
STEPS=5 TASK="Put the red cube in the box." bash hardware/so101_lab.sh policy test2
```

11 Record demos

40 to 60 min, needs the leader

1. Pick **one** simple task and **one** fixed sentence, and use exactly that sentence every time.

2. Run:

```
TASK="Put the red cube in the box." bash hardware/so101_lab.sh record cube_box 50
```

3. **Each episode:**
   - 20 s to do the task with the leader.
   - 10 s to reset: put the cube back at a slightly different spot inside the taped area, and use the leader to move the arm back to rest. The follower keeps copying the leader during the reset.
   - About 30 s of saving.
4. **Keys** (these work on the Jetson's own screen, not over SSH):
   - **right arrow:** done early, go to the reset
   - **left arrow:** redo this episode
   - **Esc:** stop and save
5. **What makes a good demo:**
   - The same start pose every time.
   - Smooth, steady moves.
   - Finish the task every time.
   - If you drop the cube or bump something, press the left arrow and redo it.
6. **If recording stops early,** the episodes saved so far are kept. To add more episodes to the same dataset:

```
RESUME=1 TASK="Put the red cube in the box." bash hardware/so101_lab.sh record
cube_box 20
```

12 Pack up

10 min

1. Copy the demos and calibration files to the USB drive. Use the drive's real name from `ls /media/$USER`:

```
cp -r ~/.cache/huggingface/lerobot/local ~/.cache/huggingface/lerobot/calibration
/media/$USER/DRIVE/
```

2. Switch off the arm power supplies, then unplug the USB cables.
3. Shut the Jetson down from the menu. Wait until it's fully off before unplugging it.
4. **Fill in the order list in Part 4 together before we leave.**

Part 4. **Inventory and order list (fill in at the lab)**

Item	Need	Have?	Notes / what to order
Follower arm, `motors follower` shows 6 of 6	1	☐	
Leader arm, `motors leader` shows 6 of 6	1	☐	Needed for demos, and for the Isaac sim teleop at home
Driver board on each arm	2	☐	
Power supply per arm, voltage matches the servo label	2	☐	
USB cable per arm (driver board to Jetson)	2	☐	
Table clamps	2	☐	
USB 3.0 cameras (one front or top view, one wrist)	2	☐	USB 3.0 only (Dr. Liu)
Camera mounts (tripod or clamp, wrist mount)	2	☐	
Calibration files from the lab	optional	☐	
Monitor (DisplayPort or HDMI) and outlets at the arm	1	☐	
Task objects (cubes, box)		☐	

Decide before we leave

- Do we buy our own leader and follower for home, so work isn't limited to lab hours?

- How many cameras, and which model?

- Anything broken or missing from the table above.

Notes

Part 5. **Troubleshooting**

What you see	Likely cause	What to do
`permission denied` on `/dev/ttyACM0`	not in the dialout group	`sudo chmod 666 /dev/ttyACM0` (until replug)
`Could not connect on port`	wrong port, or the USB cable is unplugged	`bash hardware/so101_lab.sh ports`, then `export FOLLOWER_PORT=...`
`motors`: "No servo answers"	arm power off, or the board-to-first-servo cable is loose, or the jumpers aren't on B	plug in the supply, reseat the cable, check the jumpers
`motors`: some IDs "MISSING"	a loose cable in the servo chain, or a broken servo	reseat the cable going into the first missing servo
`motors`: "NOT an STS3215"	a different servo type	write it down. Our software expects STS3215
`motors`: voltage warning	wrong or weak power supply	check the labels and swap the supply
"Press ENTER to use provided calibration file..."	a calibration file with this name already exists	Enter uses it. Type `c` and Enter to calibrate again
teleop: a joint moves the wrong way or jumps	bad calibration	redo Step 7 for that arm
camera shows 480M	USB 2 port, hub, or cable	use one of the Jetson's own ports and a USB 3 cable
preview is black	lens cap, or the wrong device	use the `-video-index0` path from `check`
"Record loop is running slower ..." during the policy run	the Jetson is busy planning	normal. Lower it with `STEPS=5`
program killed, or out of memory	the browser or VS Code is open	close them and try again
the arm does something unsafe		**unplug the follower's power supply**
the arm drops when a program ends	the motors switch off on exit	hold the arm before it ends

Part 6. **After Thursday**

- Copy the demos to the RTX 4090 and fine-tune SmolVLA on them. That takes a few hours, and WSL2 on Windows should be enough for this step.
- Copy the finished checkpoint folder to the Jetson.
- On the next lab visit, set up the arm and cameras exactly like Thursday, then run:

  ```
  TASK="Put the red cube in the box." bash hardware/so101_lab.sh policy run1
  /path/to/checkpoint
  ```

  Count the successes out of 10 tries.

Numbers **measured on our board**

2026-10-20, MAXN SUPER, clocks not pinned

SmolVLA base, 640x480 frames	Time to plan 50 moves (p50)
2 cameras, 10 steps (default)	1129 ms
1 camera, 10 steps	950 ms
2 cameras, 5 steps	680 ms
1 camera, 5 steps	586 ms

- **Memory and power:** 0.9 GiB of GPU memory, about 11 W board power.
- **In-between steps:** about 18 ms each, out of a 33 ms budget at 30 Hz.
- **Bottleneck:** the CPU, not the GPU. The CPU is busy the whole time, handing the GPU thousands of tiny pieces of work. TensorRT is the long-term fix, and that is our project.
- **Other apps don't change the planning time.**
  - One open desktop app used about 1.1 CPU cores, and the desktop itself about 0.4.
  - Adding 1 or 2 more cores of load left the planning time at 1056 to 1126 ms, within normal run-to-run variation.
  - Planning runs on a single CPU thread that already has its own core at almost full clock speed.
- **To recheck:** run `vla` first, then `python3 hardware/hello_smolvla.py`. Add `--camera /dev/v4l/by-id/...` to use real camera frames.

Source: docs/guides/lab-day-2026-10-22.md. Scripts: hardware/so101_lab.sh and hardware/hello_smolvla.py.